

Даалгавар 8 - Монгол орчуулга

Эх сурвалж: “Даалгавар 8.pdf” файлын 17-25-р хуудасны орчуулга

Тайлбар: Томьёо, тэмдэглэгээ, файлын нэр болон командын мөрийн жишээнүүдийг ойлгомжтой байлгахын тулд эх хувилбартай нь ойролцоо хэлбэрээр хадгалав.

Програмчлал (55 оноо)

Зураг 2. OCR өгөгдлийн багц дахь 10 үсэг тус бүрээс санамсаргүй сонгосон 10 зураг.

5 Даалгавар

Энэ даалгаврын зорилго нь нэг нууц давхаргатай нейрон сүлжээ хэрэгжүүлж, зургуудыг ангилах юм.

6 Өгөгдлийн багц

Энэ даалгаварт бид Оптик тэмдэгт таних (OCR) өгөгдлийн багцын дэд хэсгийг ашиглана. Анхны өгөгдөл нь гар бичмэлийн 26 үсгийн зурагтай; харин бид зөвхөн “a”, “e”, “g”, “i”, “l”, “n”, “o”, “r”, “t”, “u” гэсэн 10 үсгийг ашиглана. Гарын авлагад буй жижиг өгөгдлийн багц нь ангилал тус бүрт 60 жишээтэй (50 нь сургалт, 10 нь баталгаажуулалт). Мөн ангилал тус бүрт 600 жишээтэй дунд хэмжээний өгөгдлийн багц дээр (500 нь сургалт, 100 нь баталгаажуулалт) таны кодыг шалгана. Зураг 2-т энэ өгөгдлийн багцаас авсан зарим үсгийн 10 санамсаргүй жишээг үзүүлсэн болно (энэ даалгаварт ангилах үсгүүдтэй яг ижил биш байж болно).

Өгөгдлийн файлын бүтэц: жижиг, дунд, том бүх өгөгдлийн багц нь train болон validation гэсэн хоёр CSV файлаас бүрдэнэ. Мөр бүр 129 баганатай байна. Эхний багана нь шошгыг агуулна; 2-129 дүгээр баганууд нь 16 x 8 хэмжээтэй зургийн пикселийн утгуудыг мөрийн дагуу (row-major) хадгална. Шошго 0 нь “a”, 1 нь “e”, 2 нь “g”, 3 нь “i”, 4 нь “l”, 5 нь “n”, 6 нь “o”, 7 нь “r”, 8 нь “t”, 9 нь “u” үсэгтэй тохирно.

Анхны зургууд нь grayscale биш, хар-цагаан тул пикселийн утгууд зөвхөн 0 эсвэл 1 байна. Гэхдээ таны код [0, 1] муж дахь дурын утгыг хүлээж авдаг байхаар бичигдэх ёстой. Зураг 2 дээрх дүрсүүдийг эдгээр пикселийн утгыг .png файл болгон хөрвүүлж үзүүлсэн. Энд ямар нэг feature engineering хийгдээгүй; харин таны байгуулах нейрон сүлжээ тэмдэгт танихад хэрэгтэй шинжүүдийг өөрөө сурна.

7 Загварын тодорхойлолт

Энэ даалгаварт та нэг нууц давхаргатай нейрон сүлжээ хэрэгжүүлнэ. Нууц давхаргад sigmoid activation function, гаралтын давхаргад softmax ашиглана. Оролтын вектор x -ийн уртыг M , нууц давхаргын зангилааны тоог D , гаралтын $\hat{y}$ векторыг K ангиллын магадлалын тархалт гэж үзье. Өөрөөр хэлбэл гаралтын векторын $\hat{y}_k$ бүрэлдэхүүн нь x нь k ангилалд хамаарах магадлалыг илэрхийлнэ.

Энэ загварыг илүү шахмал хэлбэрээр бичихийн тулд $x_0 = 1$ гэсэн bias шинж оролтод, мөн $z_0 = 1$ гэсэн тогтмол утгыг нууц давхаргад авъя. Ийм тохиолдолд бид хоёр параметрийн матрицтай байна: $\alpha \in \mathbb{R}^{D \times (M+1)}$ болон $\beta \in \mathbb{R}^{K \times (D+1)}$. Матриц бүрийн нэмэлт 0-р багана (өөрөөр хэлбэл $\alpha_{:,0}$ болон $\beta_{:,0}$) нь bias параметруудийг агуулна.

$$a_j = \sum_{m=0}^M \alpha_{j,m} x_m$$

$$z_j = \sigma(a_j) = 1 / (1 + \exp(-a_j))$$

$$b_k = \sum_{j=0}^D \beta_{k,j} z_j$$

$$\hat{y}_k = \text{Softmax}(b) = \exp(b_k) / \sum_{l=1}^K \exp(b_l)$$

Сургалтын үед ашиглах зорилтот функц нь $D = \{(x^{(i)}, y^{(i)})\}$ сургалтын өгөгдлийн дундаж кросс-энтропи юм:

$$J(\alpha, \beta) = -(1/N) \sum_{i=1}^N \sum_{k=1}^K y_k^{(i)} \log(\hat{y}_k^{(i)}) \quad (9)$$

Тэгшитгэл (9)-д J нь α ба β параметруудийн функц юм. Учир нь $\hat{y}_k^{(i)}$ нь $x^{(i)}$ -д нейрон сүлжээг хэрэгжүүлсний гаралт бөгөөд $x^{(i)}$, α , β -ээс хамаарна. Энд $\hat{y}_k^{(i)}$ ба $y_k^{(i)}$ нь тус тус $\hat{y}^{(i)}$ болон $y^{(i)}$ -ийн k дахь бүрэлдэхүүнүүд болно.

Сургалтын үед энэ зорилтот функцийг стохастик градиент буулт (SGD)-оор оновчлоно. Жишээ бүр дээрх параметрийн градиентийг backpropagation-аар тооцно. SGD хийх бүрд сургалтын жишээнүүдийг өгөгдсөн shuffle функцээр хольж, ероч-ийн дугаарыг санамсаргүй үр (random seed) болгон дамжуулах ёстой. SGD нь зорилтот функцэд бага зэрэг өөрчлөлт оруулна. Учир нь энэ үед бүх өгөгдлийн багц дээр биш, зөвхөн тухайн i жишээ дээр “нийлбэрлэж” байна:

$$J_{\text{SGD}}(\alpha, \beta) = - \sum_{k=1}^K y_k^{(i)} \log(\hat{y}_k^{(i)}) \quad (10)$$

Та параметруудийг дараах танил SGD шинэчлэлтийн дүрмээр шинэчилнэ:

$$\alpha_{t+1} \leftarrow \alpha_t - \gamma (\partial J_{\text{SGD}}(\alpha_t, \beta_t) / \partial \alpha_t) \quad (11)$$

$$\beta_{t+1} \leftarrow \beta_t - \gamma (\partial J_{\text{SGD}}(\alpha_t, \beta_t) / \partial \beta_t) \quad (12)$$

Энд γ нь сургалтын хурд (learning rate), α_t ба β_t нь t алхам дахь α ба β -ийн утгууд (ижил утгаар α_{t+1} , β_{t+1} нь дараагийн алхмын утгууд) болно.

7.1 Эхлүүлэлт

Гүн сүлжээг ашиглахын тулд жин болон bias-уудыг эхлээд инициалчлах шаардлагатай. Ихэвчлэн үүнийг санамсаргүй эхлүүлэлт эсвэл өөр сургалтын процедураас гарсан утгаар хийдэг. Энэ даалгаварт та дараах хоёр төрлийн эхлүүлэлтийг дэмжинэ:

RANDOM: жинүүдийг -0.1-ээс 0.1 хүртэлх жигд тархалтаас санамсаргүйгээр инициалчилна. Bias параметруудийг 0-ээр инициалчилна.

ZERO: бүх жинг 0-ээр инициалчилна.

Та дээрх хоёр эхлүүлэлтийн аль алийг нь дэмжих ёстой.

8 Хэрэгжүүлэлт

Нэг нууц давхаргатай нейрон сүлжээ ашиглан оптик тэмдэгт танигч хэрэгжүүлэх neuralnet.py програм бич. Таны програм сургалтын өгөгдөл дээр загварын параметруудийг сургах, ероч бүрийн төгсгөлд сургалт болон

баталгаажуулалтын өгөгдөл дээрх кросс-энтропийг тайлагнах, мөн сургалт дууссаны дараа хоёр өгөгдлийн багц дээрх таамаглал ба алдааны хувь хэмжээг файлд бичих ёстой.

Таны хэрэгжүүлэлт дараах шаардлагуудыг хангана:

- Нууц давхаргад sigmoid activation function, гаралтын давхаргад softmax ашиглаж, гаралт нь зөв магадлалын тархалт байх ёстой.
- Нууц давхаргын зангилааны тоог командын мөрийн параметрээр тохируулдаг байх ёстой.
- 7.1-р хэсэгт тайлбарласан хоёр төрлийн эхлүүлэлтийг командын мөрийн параметрээр сонгон дэмждэг байх ёстой.
- Нэг нууц давхаргатай нейрон сүлжээний параметруудийг оновчлохдоо SGD ашиглах ёстой. Epoch-ийн тоог мөн командын мөрийн параметрээр өгнө.
- Learning rate-ийг командын мөрийн параметрээр тохируулна.
- SGD шинэчлэлтийг өгөгдсөн shuffle функцээр хольсон сургалтын өгөгдөл дээр хийнэ. Epoch бүрт өмнөх epoch-ийн хольсон өгөгдлийг биш, эх файлын анхны өгөгдлийг дахин shuffle хийх ёстой.
- Оролтын өгөгдлийн гаралтын шошгын муж үргэлж {0, 1, ..., 9} байна гэж үзэж болно. Үүнээс бусад бүх зүйлийг код дотроо хатуу тогтоож болохгүй. Бид таны программыг өөр өөр жишээ агуулсан хэд хэдэн өгөгдлийн багц дээр автоматаар шалгана.
- Хэрэв гаралтын $\hat{y}$ давхаргад тэнцэл гарвал хамгийн бага индексийг шошго болгон сонго. Зөв NumPy функц хэрэглэвэл нэмэлт tie-breaking код бичих шаардлагагүй.
- Ямар нэгэн machine learning сан ашиглаж болохгүй. NumPy ашиглаж болно.

8.1 Командын мөрийн аргууд

Автомат үнэлэгч дараах хэлбэрийн командыг ажиллуулж, үүсгэсэн файлуудын гаралтыг шалгана:

```
$ python3 neuralnet.py [args...]
```

Энд [args...] нь нийт есөн аргументын орлуулагч юм: <train input> <validation input> <train out> <validation out> <metrics out> <num epoch> <hidden units> <init flag> <learning rate>. Эдгээрийн утга дараах байдалтай:

Параметр	Тайлбар
<train input>	Сургалтын оролтын .csv файлын зам.
<validation input>	Баталгаажуулалтын оролтын .csv файлын зам.
<train out>	Сургалтын өгөгдөл дээрх таамаглалыг бичих .labels гаралтын файлын зам.
<validation out>	Баталгаажуулалтын өгөгдөл дээрх таамаглалыг бичих .labels гаралтын файлын зам.
<metrics out>	Train/validation error зэрэг хэмжигдэхүүнийг бичих .txt файлын зам.
<num epoch>	Backpropagation сургалтын бүх өгөгдлөөр хэдэн удаа явахыг заасан бүхэл тоо.
<hidden units>	Нууц зангилааны тоог заасан эерэг бүхэл

	тоо.
<init flag>	1 эсвэл 2 утгатай бүхэл тоо. 1 бол RANDOM, 2 бол ZERO эхлүүлэлт. Аль ч тохиолдолд bias-уудыг 0-ээр инициалчилна.
<learning rate>	SGD-ийн learning rate-ийг заасан бодит тоо.

Жишээ нь Python хэрэгжүүлэлт дараах байдлаар ажиллана. Энэ жишээнд small өгөгдөл, 4 нууц зангилаа, 2 epoch, zero initialization, 0.1 learning rate ашиглаж байна:

```
python3 neuralnet.py small_train.csv small_validation.csv \
small_train_out.labels small_validation_out.labels \
small_metrics_out.txt 2 4 2 0.1
```

Python-ын builtin argparse package ашиглан neuralnet.py дотор эдгээр аргументуудыг задлах кодыг өгсөн.

8.2 Гаралт: Labels файлууд

Таны програм хоёр .labels гаралтын файл бичих ёстой: нэг нь сургалтын өгөгдөл (<train out>), нөгөө нь баталгаажуулалтын өгөгдөл (<validation out>) дээрх загварын таамагласан шошгуудыг агуулна. Жишээ бүрийн таамаглалыг шинэ мөрөнд нэг нэгээр нь хэвлэнэ. Шинэ мөр үүсгэхдээ ашиглана.

Автомат үнэлэгч таны гаралтыг лавлагаа хэрэгжүүлэлтийн гаралттай яг ижил эсэхээр шалгана. Иймээс шошгоны файл дахь мөр бүр яг таарч байх ёстой. neuralnet.py дотор зөв форматтай labels гаралт бичих кодыг өгсөн.

Анхаарах зүйл: Та таамагласан шошгоо анхны сургалтын өгөгдөлтэй ижил бүхэл тоон кодоор гаргах ёстой. Мөн оролтын файльтай адил дарааллын төгсгөлд нэг хоосон мөр нэмж оруулах хэрэгтэй.

8.3 Гаралт: Metrics файл

Дараах хэмжигдэхүүнүүдийг тайлагнах файл үүсгэ:

- cross entropy: Epoch бүрийн дараа сургалтын өгөгдөл дээрх дундаж кросс-энтропи `crossentropy(train)` болон баталгаажуулалтын өгөгдөл дээрх `crossentropy(validation)`-ийг тайлагнана. Эдгээр хоёр утгыг epoch-ийн дугаартай хамт хэвлэнэ. Жишээлбэл хоёр дахь epoch-ийн дараах утгууд epoch=2 гэсэн утгвартай байна. Нийт хэвлэх сургалтын loss-ийн тоо нь num epoch-той, validation loss-ийн тоо мөн түүнтэй адил тэнцүү байна.
- error: Сүүлчийн epoch дууссаны дараа эцсийн сургалтын алдаа `error(train)` болон баталгаажуулалтын алдаа `error(validation)`-ийг тайлагнана.

Жижиг өгөгдлийн багц дээрх жишээ гаралт дараах хэлбэртэй байна:

```
epoch=1 crossentropy(train): 2.1415670910950144
epoch=1 crossentropy(validation): 2.1502231738985618
epoch=2 crossentropy(train): 1.8642629963917074
epoch=2 crossentropy(validation): 1.8780601379038728
error(train): 0.73
```

```
error(validation): 0.72
```

Формат нь дээрхтэй яг ижил байх ёстой. epoch ба дугаарын хооронд = тэмдэг байна, харин зай байхгүй. Epoch-ийн дугаарын дараа ганц зай, metric-ийн өмнөх двоеточийн дараа мөн ганц зай байна. Мөр бүр Unix төрлийн шинэ мөрийн төгсгөлтэй байна. Эдгээр metrics-ийг зөв форматаар бичих кодыг neuralnet.py дотор өгсөн.

8.4 Unit test-үүд

Кодоо алдаагүй эсэхийг шалгахад туслахын тулд tests.py гэсэн unit test файлыг өгсөн. Энэ нь бүрэн гүйцэд шалгалт биш боловч хэрэгжүүлэлтийн аль хэсэгт алдаа байж болохыг олоход тусална. Эдгээр тестүүдийг давлаа гээд Gradescope дээр бүтэн оноо авна гэсэн баталгаа биш. Харин аль функц буруу байгааг илрүүлэхэд хэрэг болно. Эдгээр тестүүдийг өөрчилж болохгүй.

Unit test-үүдийг дараах командуудаар ажиллуулна:

```
Нэг тест ажиллуулах:  python -m unittest tests.TestRandomInit.test_shape
Нэг бүлэг тест:       python -m unittest tests.TestRandomInit
Бүх тест:              python -m unittest tests
```

Хэрэв дээрх командууд алдаа өгвөл python-ыг python3 болгон сольж үзээрэй.

9 Gradescope-д илгээх

Та зөвхөн neuralnet.py файлаа Gradescope-д илгээх ёстой. Бусад бүх файл устгагдана. Хэрэгжүүлэлтийн файлаа өөр нэрээр бүү хадгал. Тэгвэл автомат үнэлэгч зөв илрүүлж ажиллуулж чадахгүй болно.

Autograder-ийн гаргасан мэдээллийг анхааралтай унш. Энэ програмчлалын даалгаварт бид түгээмэл гардаг зарим буруу хэрэгжүүлэлтүүдийг тусгайлан илрүүлэх тестүүд бэлдсэн. Тиймээс Gradescope-ийн autograder-ийн гаралтыг алдаагаа засахад ухаалгаар ашигла.

Анхаарах зүйл: Энэ даалгаварт deadline-оос өмнө 10 хүртэл удаа илгээх боломжтой, гэхдээ зөвхөн хамгийн сүүлийн илгээлт үнэлэгдэнэ.

10 Модульд суурилсан нейрон сүлжээний хэрэгжүүлэлт

10.1 Хэрэгжүүлэх модульд суурилсан арга

Модульд суурилсан automatic differentiation (AD) нь deep learning сангуудыг хөгжүүлэхэд удаан хугацаанд хэрэглэгдсэн арга бөгөөд энэ даалгаварт таны дагахыг зөвлөж буй хэрэгжүүлэлтийн арга юм. Dynamic neural network package гэдэг нь computation graph-ийг ажиллах үед нь динамикаар тодорхойлох боломжтой сангуудыг хэлнэ. Жишээ нь Torch, PyTorch, DyNet зэрэг нь энд тайлбарлах утгаараа модульд суурилсан AD ашигладаг.

Модульд суурилсан AD-ийн гол санаа нь нейрон сүлжээний тооцооллыг давхарга бүрээр нь компонентчлох явдал юм. Давхарга бүрийг computation graph дахь олон зангилааг (түүний дэд хэсгийг) нэг вектор утгатай зангилаанд нэгтгэж байна гэж ойлгож болно. Ийм вектор зангилааг module гэж нэрлэнэ (Python дээр энэ нь class-той таарна). Module бүр дараах чадвартай байх ёстой:

1. Оролтын $a = [a_1, \dots, a_A]$ -г авч, ялгаварлах боломжтой f функцээр гаралтын $b = [b_1, \dots, b_B]$ -г тооцох forward computation. Өөрөөр хэлбэл $b = f(a)$.
2. Гаралтын gradient болох $g_b = \partial J / \partial b = [dJ / db_1, \dots, dJ / db_B]$ -г өгөгдсөн үед оролтын gradient болох $g_a = \partial J / \partial a = [dJ / da_1, \dots, dJ / da_A]$ -г тооцох backward computation. Энд J нь бүх computation graph-ийн төгсгөлийн бодит утгат гаралт юм. Энэ нь chain rule-ээр хийгдэнэ.

10.1.1 Модулиудын тодорхойлолт

Манай хэрэгжүүлэлтэд нейрон сүлжээний модулиуд нь Linear layer болон Sigmoid layer байна. Softmax ба Cross-Entropy-г мөн модуль болгож болох ч энгийн байлгахын тулд тэдгээрийг функц хэвээр үлдээнэ (хүсвэл та модуль болгож болно). Module бүр forward method $b = \text{*.FORWARD}(a)$, мөн backward method $g_a = \text{*.BACKWARD}(g_b)$ -тай байна. Өөрөөр хэлбэл forward method нь оролт a -гаас гаралт b -г үүсгэнэ; backward method нь гаралтын gradient g_b өгөгдсөн үед оролтын gradient g_a -г буцаана. Module бүр шаардлагатай завсрын утгуудыг хадгалж болно. Жишээ нь Linear layer-ууд α, β жингийн матрицуудыг хадгална.

Ялангуяа linear module-ийн хувьд оролт, гаралтын gradient-үүд module руу орж гардаг боловч жингийн матрицтай холбоотой g_α болон g_β gradient-үүдийг гадагш дамжуулах шаардлагагүй. Энэ нь encapsulation гэсэн объект хандалтат зарчимтай нийцнэ: g_α болон g_β нь зөвхөн өөр өөрийн linear layer-т хэрэгтэй тул тухайн layer дотроо хадгална. Дараа нь тэдгээрийг STEP method ашиглан SGD шинэчлэлт хийхэд хэрэглэнэ. (Эсвэл энэ даалгаварт шинэчлэлтийг жишээ бүр дээр хийдэг тул BACKWARD дотор нь шууд шинэчлэлт хийж болно. Гэхдээ үйлдлүүдийн дараалал дээр маш болгоомжтой байх ёстой.)

Хэрэв та Written Question 2-ыг хийсэн бол $\text{*.BACKWARD}(g_b)$ руу зөвхөн гаралтын gradient болох g_b -г дамжуулж байгаа хэдий ч давхаргын зарим gradient-ийг тооцоход forward pass-аас гарсан нэмэлт утгууд хэрэг болохыг анзаарах байх. Иймээс forward method-ууд дотор дараа нь backward pass-д ашиглах утгуудыг cache хийж хадгалах шаардлагатай. Starter code-д үүнийг

cache dictionary гэсэн class attribute ашиглан хийдэг; та параметрийн нэрийг key, утгыг value болгон хадгалж болно.

Мөн module руу өгч, module-оос авч буй векторуудын хэмжээст онцгой анхаараарай. А ба В хэмжээ нь тухайн module-оос хамаарна. Өөрөөр хэлбэл оролт $a \in \mathbb{R}^A$, гаралт $b \in \mathbb{R}^B$, гаралтын gradient $g_a = \nabla_a J \in \mathbb{R}^A$, оролтын gradient $g_b = \nabla_b J \in \mathbb{R}^B$ байна.

Жишээ болгон Linear module-ийн псевдокодыг өгсөн:

Linear Module

```
1: procedure FORWARD(a)
2:   Энэ давхаргын жингийн матрицаар b-г тооц.
3:   Backward pass-д хэрэгтэй завсрын утгуудыг cache хий.
4:   return b
5: procedure BACKWARD(g_b)
6:   Шаардлагатай cached утгуудыг авч ашигла.
7:   g_a-г тооц.
8:   g_a-г тооц.
9:   Дараагийн SGD шинэчлэлтэд зориулан g_a-г хадгал.
10:  return g_a
11: procedure STEP()
12:  Хадгалсан g_a gradient-ийг ашиглан  $\alpha$  жинд SGD шинэчлэлт хий.
```

10.1.2 Нейрон сүлжээнд зориулсан модульд суурилсан AD

Нэг нууц давхаргатай нейрон сүлжээ нь хэд хэдэн module-ийн нийлмэл бүтэц тул forward болон backward propagation-ийг дараах функцүүдээр тодорхойлж болно:

Алгоритм 1. Forward computation

```
1: procedure NNFORWARD(сургалтын жишээ (x, y))
2:   a = LINEAR1.FORWARD(x)           # Эхний linear layer
3:   z = SIGMOID.FORWARD(a)           # Sigmoid activation
4:   b = LINEAR2.FORWARD(z)           # Хоёр дахь linear layer
5:    $\hat{y}$  = SOFTMAX(b)                  # Softmax function
6:   J = CROSSENTROPY(y,  $\hat{y}$ )         # Cross-entropy function
7:   return J,  $\hat{y}$ 
```

Алгоритм 2. Backpropagation

```
1: procedure NNBACKWARD(y,  $\hat{y}$ )
2:    $g_J = \partial J / \partial J = 1$       # Суурь тохиолдол
3:    $g_b = \text{DSOFTMAXCROSSENTROPY}(y, \hat{y}, g_J)$ 
4:    $g_z = \text{LINEAR2.BACKWARD}(g_b)$ 
5:    $g_a = \text{SIGMOID.BACKWARD}(g_z)$ 
6:    $g_x = \text{LINEAR1.BACKWARD}(g_a)$     #  $g_x$ -г цааш ашиглахгүй
```

Эндээс авах гол санаа нь: эдгээр хоёр функцийг өөрсдийг нь нэг өөр module гэж үзэж болно. Өөрөөр хэлбэл, нэг сургалтын жишээн дээрх нейрон сүлжээний кросс-энтропи нь өөрөө ялгаварлагдах функц бөгөөд түүний оролтын gradient-ийг, гаралтын gradient-ээр нь дамжуулан тооцож чадна.

10.2 Сургалтын процедур

7-р хэсэгт тодорхойлсон нейрон сүлжээг i дахь сургалтын жишээ (x, y) -д хэрэглэе. Энд y нь жинхэнэ шошгын one-hot кодлолт байна. Манай сүлжээ $\hat{y} = h_{\{\alpha, \beta\}}(x)$ гаралтыг өгнө. Энд α ба β нь эхний болон хоёр дахь давхаргын

параметрууд, $h_{\{\alpha, \beta\}}$ нь sigmoid activation болон softmax гаралттай нэг нууц давхаргатай нейрон сүлжээ юм. Алдагдлын функц нь сөрөг кросс-энтропи: $J = \ell(\hat{y}, y) = -y^T \log(\hat{y})$. Товчлохын тулд $J = J_{\{x, y\}}(\alpha, \beta)$ гэж бичье. Энэ нь сургалтын жишээ (x, y) болон загварын параметр α, β -ээс хамаарах функц юм.

Нейрон сүлжээг сургахын тулд бид SGD хэрэглэнэ. Gradescope-ийн туршилт дээр таны программын зан төлөв ойролцоогоор детерминистик байх ёстой тул (1) epoch бүрийн эхэнд өгөгдлөө өгсөн shuffle функцээр хольж, (2) тогтмол learning rate ашиглахыг шаардсан.

Доорх алгоритмд E нь epoch-ийн тоо, γ нь learning rate байна:

Алгоритм 3. Стохастик градиент буултаар (SGD) сургах

```

1: procedure SGD(сургалтын өгөгдөл  $D_{\text{train}}$ , тестийн өгөгдөл  $D_{\text{t}}$ )
2:   RANDOM эсвэл ZERO аргаар параметр  $\alpha, \beta$ -г инициалчил.
3:   for  $e \in \{1, 2, \dots, E\}$  do
4:      $D = \text{SHUFFLE}(D_{\text{train}}, e)$ 
5:     for  $(x, y) \in D$  do
6:       Forward prop тооц:
7:        $J, \hat{y} = \text{NN.FORWARD}(x, y, \alpha, \beta)$ 
8:       Backprop-аар градиентуудыг тооц:
9:        $g_{\alpha} = \partial J / \partial \alpha, \quad g_{\beta} = \partial J / \partial \beta$  ( $\text{NN.BACKWARD}(y, \hat{y})$ -аар)
10:      SGD шинэчлэлт хий:
11:       $\alpha \leftarrow \alpha - \gamma g_{\alpha}$ 
12:       $\beta \leftarrow \beta - \gamma g_{\beta}$ 
13:      Сургалтын дундаж кросс-энтропи  $J_D(\alpha, \beta)$ -г үнэл.
14:      Тестийн дундаж кросс-энтропи  $J_{\{D_{\text{t}}\}}(\alpha, \beta)$ -г үнэл.
15:   return параметр  $\alpha, \beta$ 

```

10.3 Тестлэх процедур

Тестлэх үед бид жишээ бүрийн хамгийн өндөр магадлалтай шошгыг гаргана:

Алгоритм 4. Тест үеийн таамаглал

```

1: procedure PREDICT(шошгогүй сургалт эсвэл тестийн өгөгдөл  $D'$ )
2:   for  $x \in D'$  do
3:     Нейрон сүлжээний таамаглал  $\hat{y} = h(x)$ -г тооц.
4:     Хамгийн өндөр магадлалтай шошгыг сонго:  $l = \text{argmax}_k \hat{y}_k$ 

```